



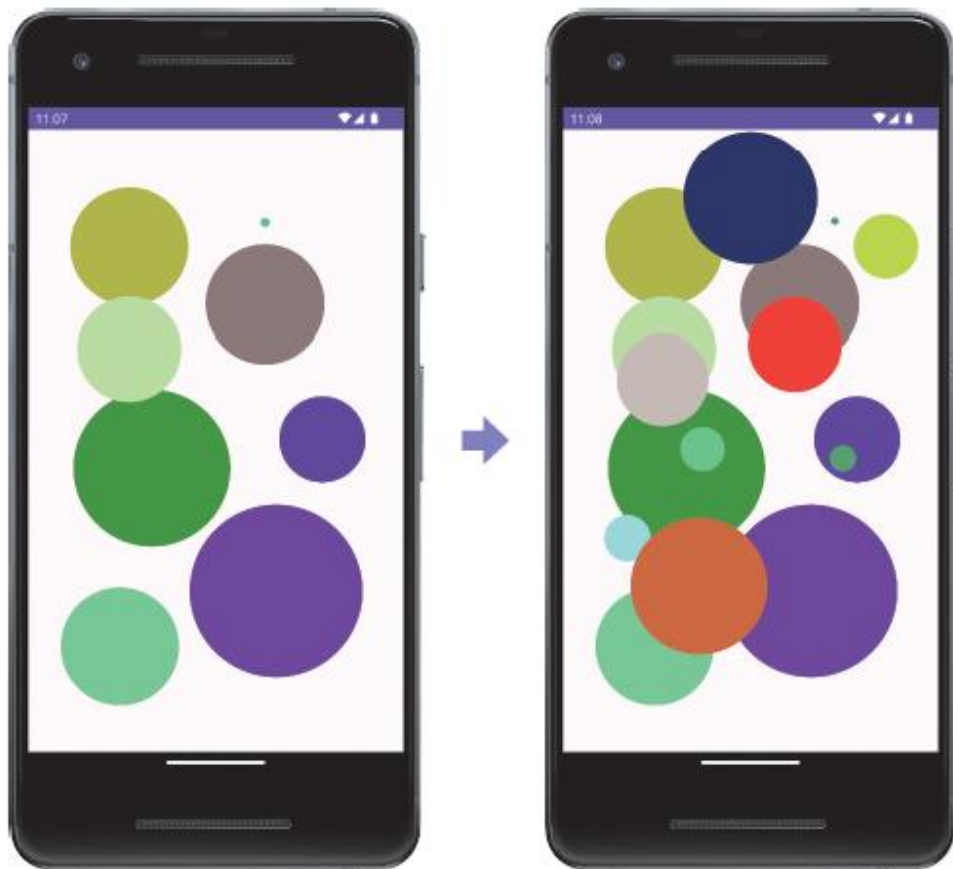
CHAP 9. 그래픽과 터치 이벤트 처리



그래픽과 터치 이벤트 처리



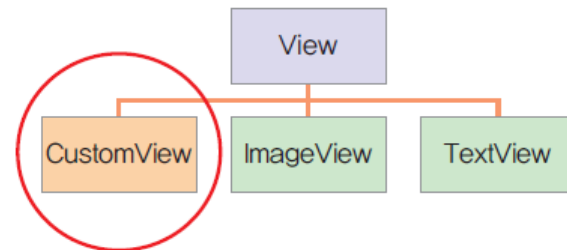
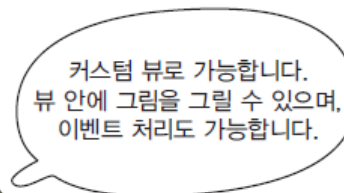
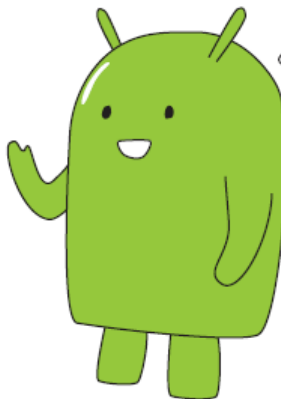
6주 목표 #1





커스텀 뷰

- 만약 기본적으로 제공되는 뷰에 만족하지 못한다면 어떤 방법이 있을까? 우리가 직접 **View** 클래스를 상속받아서 새로운 위젯을 만들 수 있을까?
- 가능하다. 안드로이드에서는 이것을 커스텀 뷰(Custom View)라고 부른다.





커스텀 뷰

- 커스텀 뷰에서 그림을 그리려면 **View** 클래스의 **onDraw()** 메소드를 오버라이드하면 된다. 커스텀 뷰 안에서 터치 이벤트를 처리하려면 **View** 클래스의 **onTouchEvent()** 메소드를 오버라이드한다.

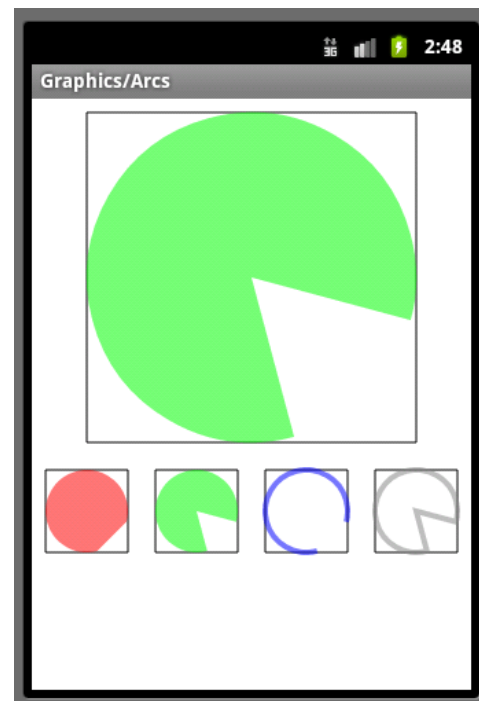
```
public class CustomView extends View {  
  
    @Override  
    protected void onDraw(Canvas canvas) {  
        // 그리기 코드 작성  
    }  
  
    @Override  
    public boolean onTouchEvent(MotionEvent event) {  
        // 터치 이벤트 처리 코드 작성  
    }  
}
```

커스텀 뷰 안에서 그림을 그리려면 **onDraw()** 메소드를 구현해야 한다.

커스텀 뷰에서 터치 이벤트를 처리할 수 있다.

안드로이드에서의 그래픽 구현 방법

- ① XML 파일로 그래픽이나 애니메이션을 정의한다. 그리는 작업은 안드로이드 시스템이 담당한다. -> 생략
- ② `onDraw()` 메소드 안에 `draw...()`와 같은 메소드를 호출하여 직접 그린다.





전체적인 구조



전체
구조

```
class MyView extends View {
```

```
    ...
```

```
    protected void onDraw(Canvas canvas) {
```

```
        Paint paint = new Paint();
```

```
        ...
```

```
    }
```

```
}
```

```
public class MainActivity extends AppCompatActivity {
```

```
    public void onCreate(Bundle savedInstanceState) {
```

```
        ...
```

```
        MyView w = new MyView(this);
```

```
        setContentView(w);
```

```
    }
```

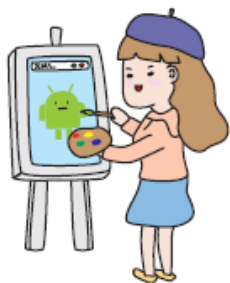
```
}
```

← 여기에 그림을 그리는 코드를
넣는다.

← MyView를 생성하고 이것을
Activity의 콘텐츠 뷰로 설정
한다.

Canvas 클래스와 Paint 클래스

Canvas 클래스



Canvas 클래스는 그림을 그리는 캔버스(화포)에 해당되는 것이다. Canvas 클래스는 도형, 이미지, 텍스트 등을 그리는 다양한 메소드들을 가지고 있다. Canvas 클래스는 그림이 캔버스의 경계를 벗어나지 않도록 잘라주는 클리핑(clipping) 기능도 제공한다. 캔버스의 크기는 `getHeight()`와 `getWidth()` 메소드를 통하여 얻을 수 있다.

Paint 클래스



Paint 클래스는 그림을 그리는 물감과 붓에 해당된다고 할 수 있다. Paint 클래스는 색상이나 선의 스타일, 채우기 스타일, 폰트, 안티에일리어싱 여부 등과 같은 그리기 속성을 가지고 있는 클래스이다. Canvas의 메소드는 항상 Paint 객체를 마지막 매개 변수로 하여 호출되어야 한다.



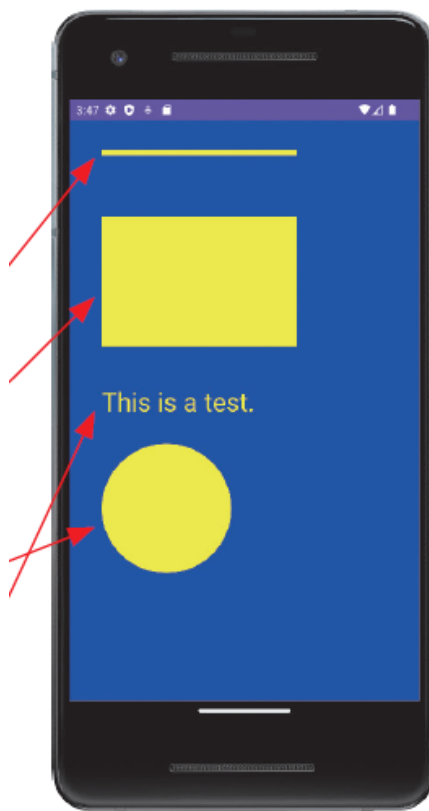
기본 도형 그리기 메소드

메소드	설명
<code>drawPoint(float x, float y, Paint paint)</code> (x, y) 위치에 점을 그린다.	점
<code>drawLine(float startX, float startY, float stopX, float stopY, Paint paint)</code> (startX, startY)부터 (stopX, stopY)까지의 직선을 그린다.	직선
<code>drawRect(float left, float top, float right, float bottom, Paint paint)</code> 좌측 상단이 (left, top)이고 우측 하단이 (right, bottom)인 사각형을 그린다.	사각형
<code>drawCircle(float cx, float cy, float radius, Paint paint)</code> 중심이 (cx, cy)이고 반지름이 radius인 원을 그린다.	원
<code>drawText(String text, float x, float y, Paint paint)</code> (x, y) 위치에 텍스트를 그린다.	텍스트



예제: 기본 도형 그리기

- 몇 개의 도형을 화면에 그려본다.



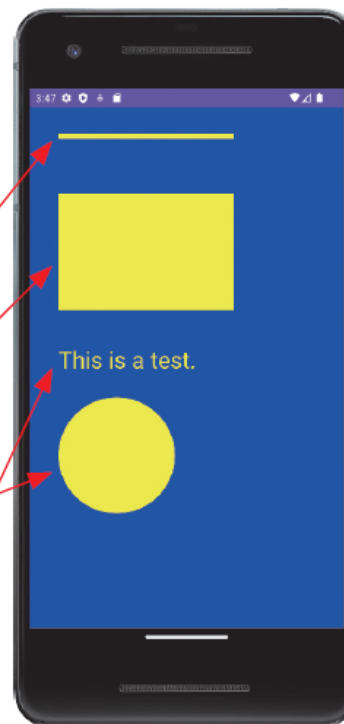


예제: 기본 도형 그리기

GraphicActivity.java

```
package kr.co.company.graphic;  
// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.
```

```
class MyView extends View {  
    public MyView(Context context) {  
        super(context);  
        setBackgroundColor(Color.BLUE);  
    }  
  
    @Override  
    protected void onDraw(Canvas canvas) {  
        Paint paint = new Paint();  
        paint.setColor(Color.YELLOW);  
        paint.setStrokeWidth(20);  
        canvas.drawLine(100, 100, 700, 100, paint);  
        canvas.drawRect(100, 300, 700, 700, paint);  
        canvas.drawCircle(300, 1200, 200, paint);  
        paint.setTextSize(80);  
        canvas.drawText("This is a test.", 100, 900, paint);  
    }  
}
```





예제: 기본 도형 그리기

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        MyView w = new MyView(this);  
        setContentView(w);  
    }  
}
```



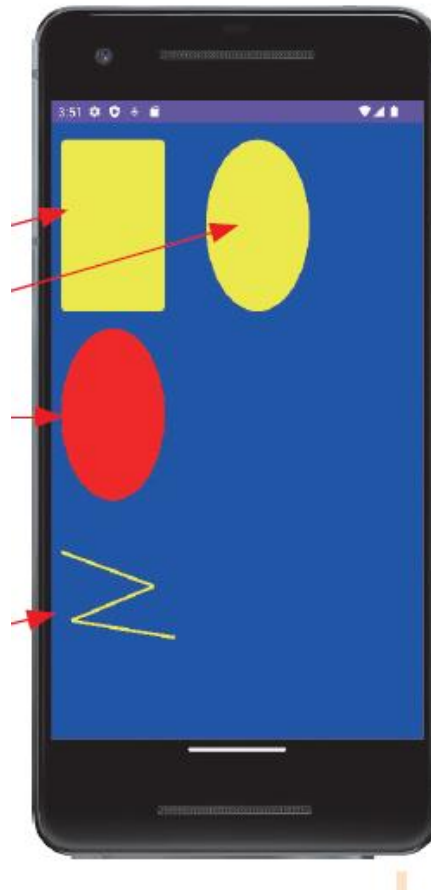
보장한 도형 그리기

메소드	설명
<code>drawRoundRect(RectF rect, float rx, float ry, Paint paint)</code> 둥근 사각형을 그리는 메소드이다. 여기서 RectF는 사각형 객체이며, rx와 ry는 사각형 모서리 부분의 반지름이다.	둥근 사각형
<code>drawOval(RectF oval, Paint paint)</code> drawOval()은 oval에 내접하는 타원을 그린다. paint의 스타일에 따라서 채워지거나 외곽선만 그려진다.	타원
<code>drawArc(RectF oval, float startAngle, float sweepAngle, boolean useCenter, Paint paint)</code> drawArc()는 oval 안에 그려지는 원호이다. startAngle에서 시작하여 sweepAngle까지 원호를 그린다. 각도의 단위는 도(degree)이며, 시계방향으로 증가한다. 만약 sweepAngle이 360보다 크면 타원이 전부 그려진다. 만약 useCenter가 참이면 원점을 포함하여 썰기(wedge) 모양으로 그려진다.	원호
<code>drawLines(float[] pts, Paint paint)</code> drawLines()는 연속된 직선들을 그린다. 각 직선은 pts 배열에서 연속된 4개의 값을 취하여 그려진다. 따라서 1개의 직선을 그리기 위해서는 배열은 적어도 4개의 값을 가지고 있어야 한다. drawLine(pts[0], pts[1], pts[2], pts[3]), drawLine(pts[4], pts[5], pts[6], pts[7]), ...이 연속되는 것과 논리적으로 동일하다.	연속된 직선
<code>drawColor(int color)</code> drawColor()는 캔버스의 전체 비트맵을 지정된 색상으로 채운다.	색상



예제: 몇 개의 기초 도형 그리기

- 복잡한 도형을 몇 개 그려보자.





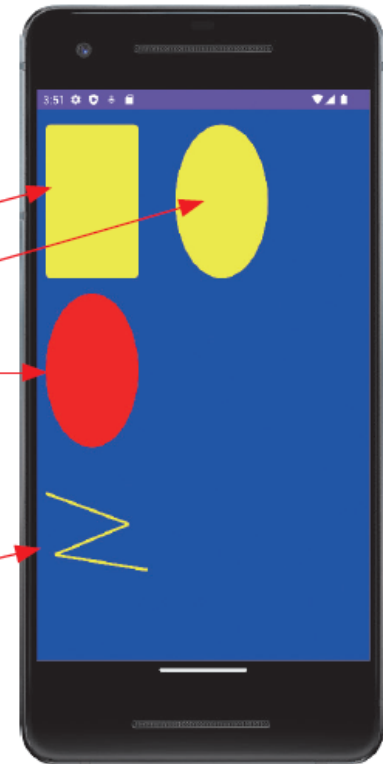
예제: 몇 개의 기초 도형 그리기

GraphicShape1Activity.java

...

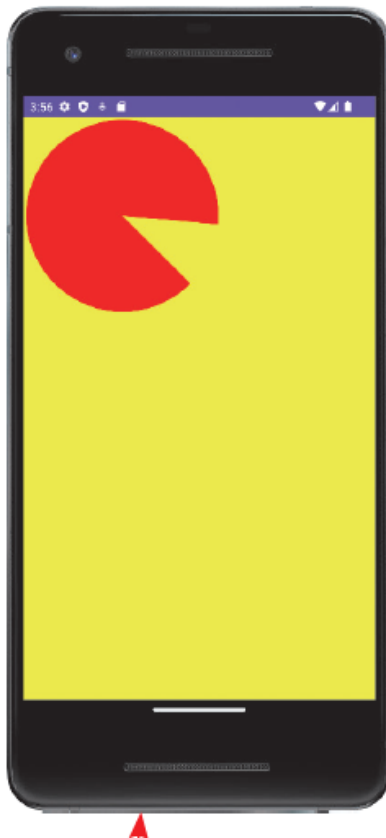
@Override

```
protected void onDraw(Canvas canvas) {  
    Paint paint = new Paint();  
    paint.setColor(Color.YELLOW);  
  
    canvas.drawColor(Color.BLUE);  
    canvas.drawRoundRect(new RectF(30,50,330,550), 15, 15, paint);  
    canvas.drawOval(new RectF(450,50,750,550), paint);  
    paint.setColor(Color.RED);  
    canvas.drawArc(new RectF(30,600,330,1100), 360, 1000, true, paint);  
    paint.setColor(Color.YELLOW);  
    float[] pts={ 30, 1250, 300, 1350, 300, 1350, 60, 1450,  
                  60, 1450, 360, 1500};  
    paint.setStrokeWidth(10);  
    canvas.drawLines(pts, paint);  
}
```



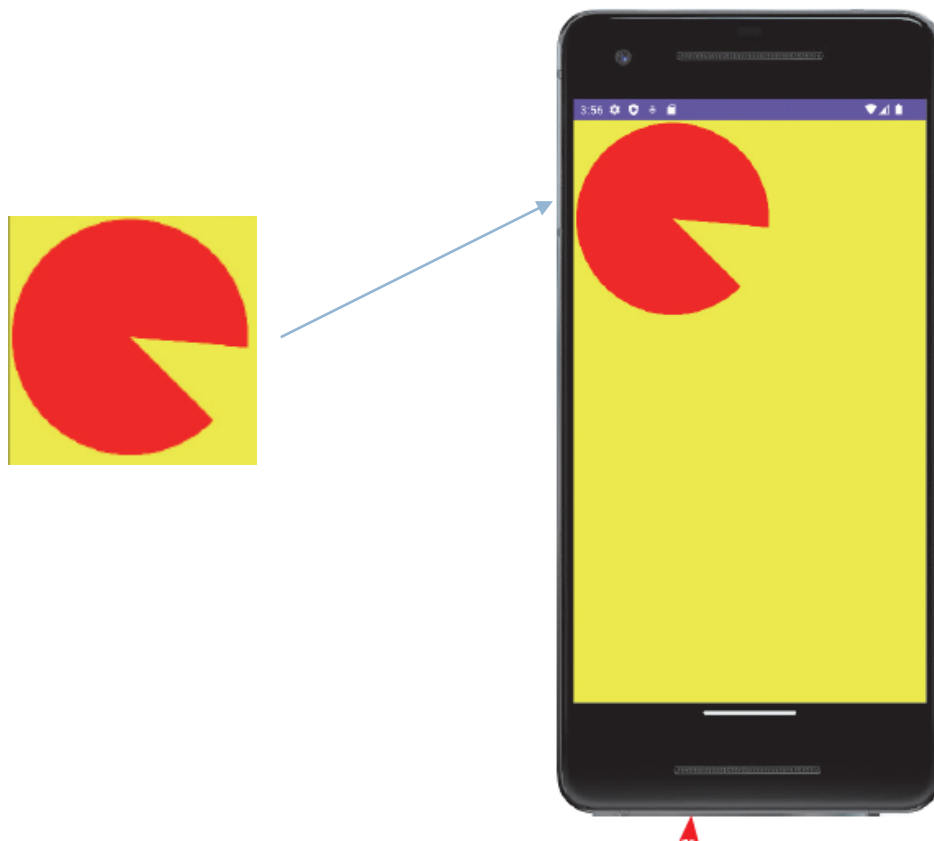
커스텀 뷰를 XML에서 참조하기

- MyView와 같은 커스텀 뷰가 정의되었을 때, 이 커스텀 뷰를 레이아웃 파일에서 참조하고자 한다면 어떻게 해야 하는가? 예제로 살펴보자.



커스텀 뷰를 XML에서 참조하기

- `kr.co.company.customview` 패키지 아래에 `CustomView.java` 파일을 생성하고 이것을 레이아웃 파일에서 참조한다.





커스텀 뷰를 XML에서 참조하기

CustomView.java

```
package kr.co.company.customview;
```

```
// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.
```

```
public class CustomView extends View {  
    public CustomView(Context context) {  
        super(context);  
        setBackgroundColor(Color.YELLOW);  
    }  
}
```

```
public CustomView(Context context, AttributeSet attrs) {  
    super(context);  
    setBackgroundColor(Color.YELLOW);  
}
```

@Override

```
protected void onDraw(Canvas canvas) {  
    Paint paint = new Paint();  
    paint.setColor(Color.RED);  
    canvas.drawArc(new RectF(10, 10, 600, 600), 45, 320, true, paint);  
}
```

XML에서 이 뷰를 참조하려면 반드시 이 생성자를 구현해야 한다.





커스텀 뷰를 XML에서 참조하기

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical" >
```

```
    <kr.co.company.customview.CustomView
```

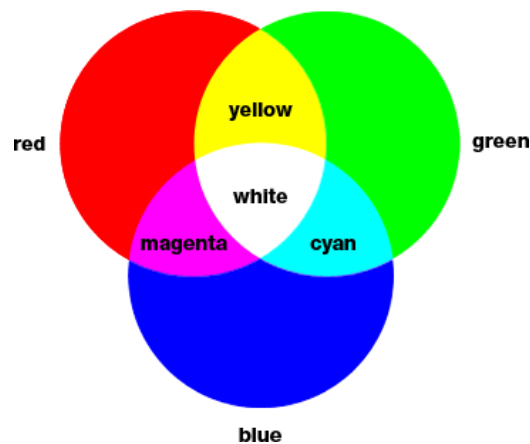
```
        android:layout_width="match_parent"
        android:layout_height="match_parent" />
```

← 커스텀 뷰의 풀네임을 적어준다.

```
</LinearLayout>
```

- 색의 3원색인 Red, Green, Blue 성분을 8비트로 표시

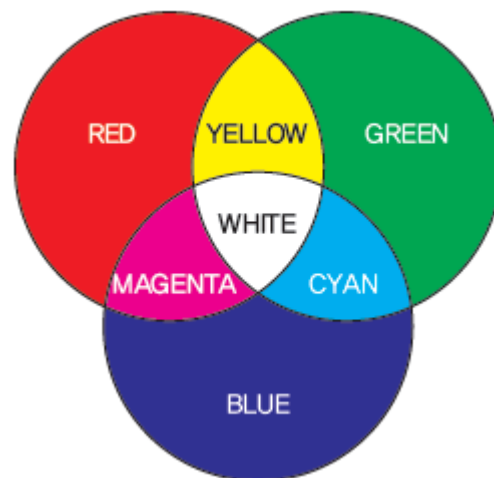
- `paint.setColor(0xFF0000);`
- `paint.setColor(Color.RED);`



- 색상은 색의 3원색인 Red, Green, Blue 성분을 8비트로 표시하여 나타낸다. 따라서 24비트면 하나의 색상을 표현할 수 있고, 24비트를 16진수로 표시하는 것이 일반적이다.

```
paint.setColor(0xFF0000);
```

```
paint.setColor(Color.RED);
```





색상 관련 메소드

메소드	설명
<code>void setColor(int color)</code> 전경색을 color로 설정한다.	전경색
<code>int getColor()</code> 현재 전경색으로 설정된 색상을 반환한다.	전경색
<code>void setAlpha(int a)</code> 0부터 255 사이의 값으로 투명도를 나타낸다.	투명도
<code>void setStrokeWidth(float width)</code> 선폭을 설정한다.	선폭
<code>void setStyle(Paint.Style style)</code> setStyle()은 각종 도형을 그리는 데 필요한 선의 스타일을 지정한다.	선 스타일



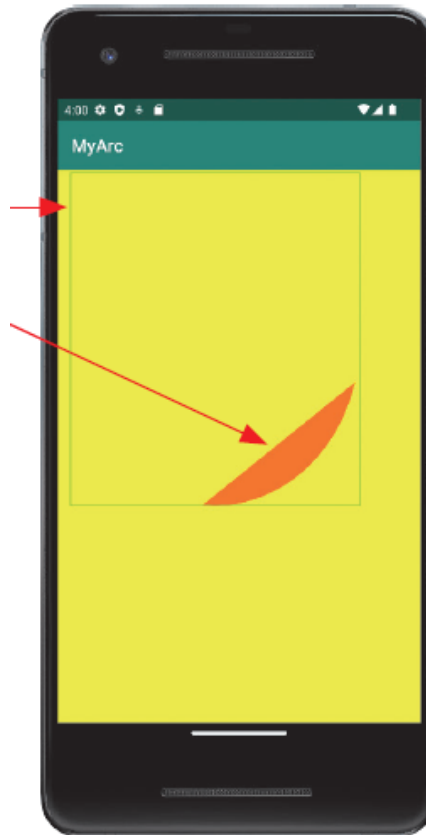
선 스타일 관련 메소드

선 스타일	설명
FILL	도형의 내부를 채운다.
FILL_AND_STROKE	도형의 내부를 채우면서 외곽선도 그린다.
STROKE	도형의 외곽선만 그린다.



예제: 원호 그리기

- 각도를 증가하면서 원호를 연속적으로 그리는 예제를 작성하여 보자.





예제:

MainActivity.java

```
package kr.co.company.myarc;  
// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.
```

```
class MyView extends View {  
    private Paint mPaints, mFramePaint;  
    private RectF mBigOval;  
    private float mStart, mSweep;  
  
    private static final float SWEEP_INC = 2;  
    private static final float START_INC = 15;
```

```
    public MyView(Context context) {  
        super(context);
```

← 시간이 많이 걸리는 작업은 생성자에서 미리 해둔다.

```
        mPaints = new Paint();  
        mPaints.setAntiAlias(true);  
        mPaints.setStyle(Paint.Style.FILL);  
        mPaints.setColor(0x88FF0000);
```

← FILL 스타일, 빨간색

```
        mFramePaint = new Paint();  
        mFramePaint.setAntiAlias(true);  
        mFramePaint.setStyle(Paint.Style.STROKE);  
        mFramePaint.setStrokeWidth(3);  
        mFramePaint.setColor(0x8800FF00);  
        mBigOval = new RectF(40, 10, 900, 1000);
```

← STROKE 스타일, 녹색,
선폭은 3 픽셀

```
    }
```



예제: 원호그리기

```
@Override
```

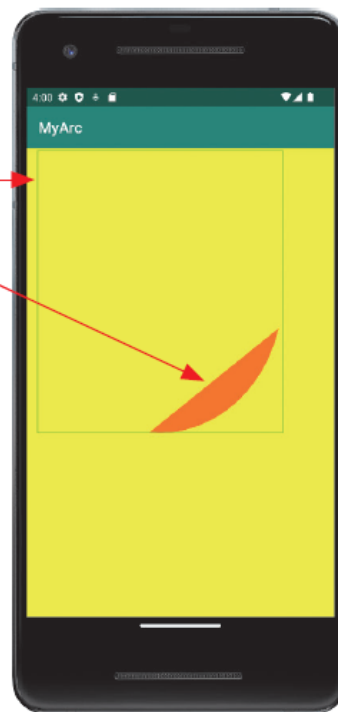
```
protected void onDraw(Canvas canvas) {  
    canvas.drawColor(Color.YELLOW);  
    canvas.drawRect(mBigOval, mFramePaint);  
    canvas.drawArc(mBigOval, mStart, mSweep, false, mPaints);  
    mSweep += SWEEP_INC;  
    if (mSweep > 360) {  
        mSweep -= 360;  
        mStart += START_INC;  
        if (mStart >= 360) {  
            mStart -= 360;  
        }  
    }  
    invalidate();  
}
```

사각형을 그린다.

mStart 각도에서 mSweep만큼의
원호를 그린다.

그리는 각도를 증가시킨다.

다시 onDraw()가 호출되게 한다.



```
public class MainActivity extends AppCompatActivity {  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(new MyView(this));  
    }  
}
```



폰트

- 안드로이드에서 폰트는 `Typeface` 클래스의 객체로 표현된다. 폰트를 생성하려면 `Typeface.create()` 메소드를 호출한다. 현재 사용되는 폰트를 변경하려면 `Paint` 클래스의 `setTypeface()`를 사용한다. 설정된 폰트를 이용하여 문자열을 화면에 그리려면 `Paint` 클래스의 `drawText()`를 호출한다.

예제: 폰트

- 폰트를 변경하면서 화면에 텍스트를 그리는 간단한 예제를 작성하여 보자.





예제: 폰트

MainActivity.java

...

@Override

protected void onDraw(Canvas canvas) {

Paint paint = new Paint();

paint.setAntiAlias(true); <----- 앤티 에일리어싱 설정

paint.setTextSize(100); <----- 폰트 설정

Typeface t;

t = Typeface.create(Typeface.DEFAULT, Typeface.NORMAL);

paint.setTypeface(t);

canvas.drawText("DEFAULT 폰트", 10, 200, paint);

t = Typeface.create(Typeface.SERIF, Typeface.ITALIC);

paint.setTypeface(t);

canvas.drawText("SERIF 폰트", 10, 300, paint);

t = Typeface.create(Typeface.SANS_SERIF, Typeface.BOLD_ITALIC);





```
    paint.setTypeface(t);  
    canvas.drawText("SANS_SERIF 폰트", 10, 400, paint);  
}  
...
```



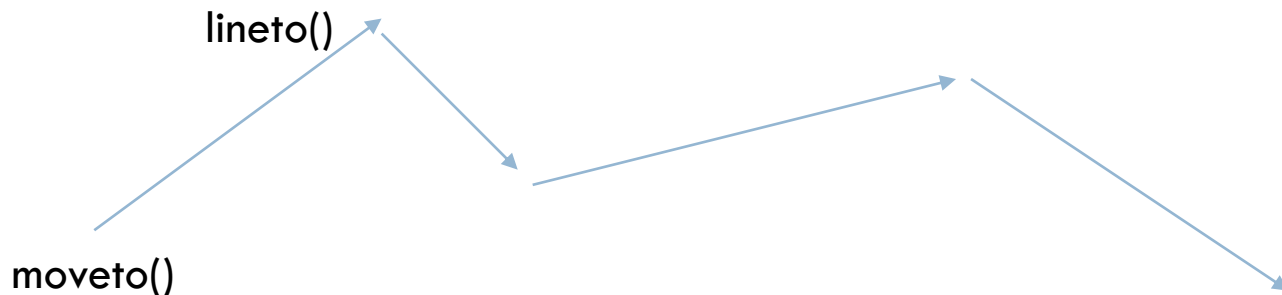
패스 그리기

- 패스(path)는 복잡한 기하학적인 경로를 표현한다. 패스는 직선과 타원, 곡선으로 이루어질 수 있다.
- 패스는 캔버스의 메소드 `drawPath()` 메소드를 이용하여 그려진다.

메소드

```
void drawPath(Path path, Paint paint)
```

주어진 paint를 이용하여서 주어진 패스를 그린다.





패스 그리기

메소드

`addArc(RectF oval, float startAngle, float sweepAngle)`

패스에 원호를 추가한다.

`addCircle(float x, float y, float radius, Path.Direction dir)`

패스에 원을 추가한다.

`addOval(RectF oval, Path.Direction dir)`

패스에 타원을 추가한다.

`addRect(float left, float top, float right, float bottom, Path.Direction dir)`

패스에 사각형을 추가한다.

`addRoundRect(RectF rect, float rx, float ry, Path.Direction dir)`

패스에 둥근 사각형을 추가한다.

`arcTo(RectF oval, float startAngle, float sweepAngle)`

패스에 원호를 추가한다.

`cubicTo(float x1, float y1, float x2, float y2, float x3, float y3)`

패스에 베지어 곡선을 추가한다.

`lineTo(float x, float y)`

패스에 직선을 추가한다.



View 클래스의 콜백 메소드

- 예를 들어서 사용자가 키패드를 누르면 View 클래스의 onKeyDown() 메소드가 호출된다. 따라서 키 이벤트를 처리하는 가장 명백한 방법은 View 클래스의 onKeyDown() 메소드를 재정의하는 것이다.



사용자가 키를 누르면
호출된다.

```
class MyView extends View
{
    ...
    boolean onKeyDown()
    {
        ...
    }
    ...
}
```



이벤트 처리 메소드

- 재정의할 수 있는 View 클래스의 이벤트 처리 메소드
 - onKeyDown(int, KeyEvent): 사용자가 키보드를 눌렀을 때 호출된다.
 - onKeyUp(int, KeyEvent): 사용자가 키에서 손을 뗐을 때 호출된다.
 - onTrackballEvent(MotionEvent): 사용자가 트랙볼을 움직였을 때 호출된다.
 - onTouchEvent(MotionEvent): 사용자가 화면을 터치했을 때 호출된다.
 - onFocusChanged(boolean, int, Rect): 뷰가 포커스를 얻거나 잃었을 경우에 호출된다.



터치 이벤트

- 최근에 스마트폰이 광범위한 인기를 얻게 된 것도 손으로 터치하여서 모든 작업을 할 수 있기 때문이다. 특히 게임에서는 터치가 아주 중요하다. 터치 이벤트도 2가지의 방식으로 처리할 수 있다.
 - 뷰의 콜백 메소드 재정의: `onTouchEvent(MotionEvent e)`를 재정의
 - 이벤트를 처리하는 객체(리스너 객체)를 생성하여 뷰에 등록하는 방법



커스텀 뷰에서 터치 이벤트 처리

- 여기서는 뷰의 메소드를 재정의하는 방법을 살펴보자.

```
class MyView extends View {
```

← 커스텀 뷰 정의

```
...
```

```
@Override
```

```
public boolean onTouchEvent(MotionEvent event) {
```

```
    x = (int) event.getX();
```

```
    y = (int) event.getY();
```

```
    ...
```

```
}
```

```
}
```

따라서 여기서는 뷰의 메소드를 재정의하는 방법을 살펴보자.



전체
구조



터치 이벤트의 종류

액션	설명
<code>ACTION_DOWN</code>	누르는 동작이 시작됨
<code>ACTION_UP</code>	누르고 있다가 땔 때 발생함
<code>ACTION_MOVE</code>	누르는 도중에 움직임
<code>ACTION_CANCEL</code>	터치 동작이 취소됨
<code>ACTION_OUTSIDE</code>	터치가 현재의 위젯을 벗어남



`ACTION_DOWN`



`ACTION_UP`

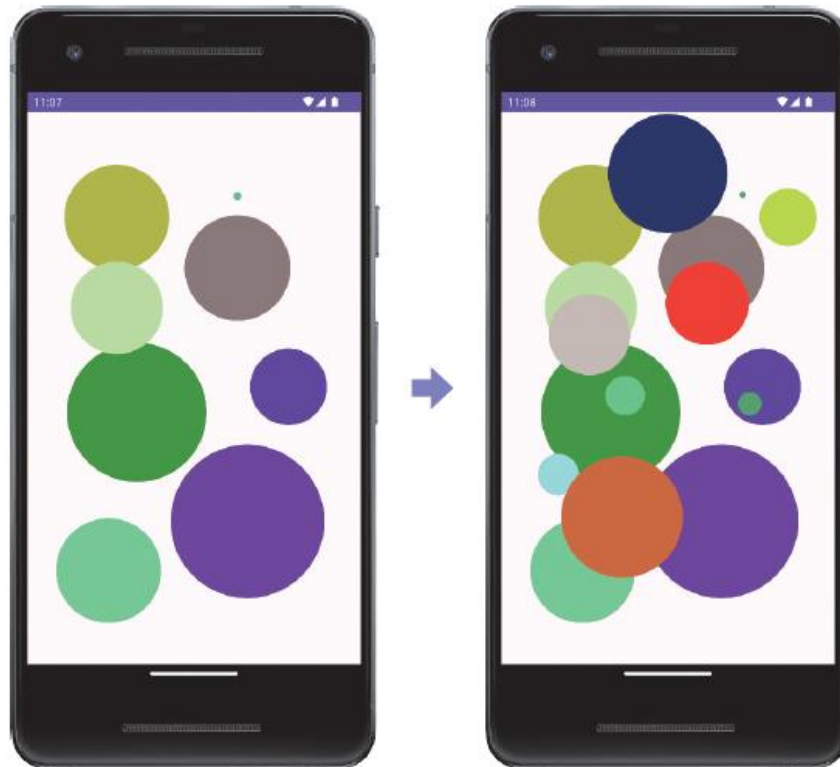


`ACTION_MOVE`



예제: 랜덤한 반지름과 색상을 가지는 원 그리기

- 화면을 터치할 때마다 랜덤한 반지름과 색상을 가지는 원이 생성되는 앱을 작성해보자.





예제: 랜덤한 반지름과 색상을 가지는 원 그리기

CustomView.java

```
public class CustomView extends View {  
    private List<Circle> circles = new ArrayList<>();  
    private Paint paint;  
  
    public CustomView(Context context) {  
        super(context);  
        paint = new Paint();  
    }  
  
    @Override  
    protected void onDraw(Canvas canvas) {  
        super.onDraw(canvas);  
        for (Circle circle : circles) {  
            paint.setColor(circle.color);  
            canvas.drawCircle(circle.x, circle.y, circle.radius, paint);  
        }  
    }  
}
```

ArrayList에 저장된 원들을 꺼내서 다시 그린다.



예제: 랜덤한 반지름과 색상을 가지는 원 그리기

```
@Override
```

```
public boolean onTouchEvent(MotionEvent event) {
```

```
    if (event.getAction() == MotionEvent.ACTION_DOWN) {
```

```
        Random random = new Random();
```

```
        float radius = random.nextInt(300);
```

```
        int color = Color.rgb(
```

```
            random.nextInt(256),
```

```
            random.nextInt(256),
```

```
            random.nextInt(256)
```

```
        );
```

```
        float x = event.getX();
```

```
        float y = event.getY();
```

```
        circles.add(new Circle(x, y, radius, color));
```

```
        invalidate(); // 화면 다시 그리도록 요청
```

```
        return true;
```

```
    }
```

```
    return super.onTouchEvent(event);
```

```
}
```

뷰의 터치 이벤트를 재정의한다.

원의 반지름과 색상은 난수로 결정
된다.



예제: 랜덤한 반지름과 색상을 가지는 원 그리기

```
private class Circle {  
    float x, y, radius;  
    int color;  
  
    Circle(float x, float y, float radius, int color) {  
        this.x = x;  
        this.y = y;  
        this.radius = radius;  
        this.color = color;  
    }  
}
```

원을 클래스로 정의한다.



```
}
```



예제: 랜더한 바지름과 색상을 가지는 원 그리기

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        CustomView w = new CustomView(this);  
        setContentView(w);  
    }  
}
```



예제: 그림판 프로그램 #1

- 사용자로부터 터치를 입력받아서 화면에 곡선을 그리는 애플리케이션을 작성해보자.





예제: 그림판 프로그램 #1

MainActivity.java

```
package kr.co.company.singletouch;

// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(new SingleTouchView(this, null));
    }

}
```



예제: 그림판 프로그램 #1

SingleTouchView.java

```
package kr.co.company.singletouch;

// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.

public class SingleTouchView extends View {
    private Paint paint = new Paint();
    private Path path = new Path();

    public SingleTouchView(Context context, AttributeSet attrs) {
        super(context, attrs);

        paint.setAntiAlias(true); ← 선분을 매끄럽게 그리기 위하여
                                   ← 안티 에일리어싱을 설정한다.
        paint.setStrokeWidth(10f); ← 선분의 두께를 10으로 한다.
        paint.setColor(Color.BLUE);
        paint.setStyle(Paint.Style.STROKE);
        paint.setStrokeJoin(Paint.Join.ROUND);
    }

    @Override
    protected void onDraw(Canvas canvas) {
        canvas.drawPath(path, paint); ← 현재까지의 경로를 모두 그린다.
    }
}
```



예제: 그림판 프로그램 #1

```
@Override
```

```
public boolean onTouchEvent(MotionEvent event) {
```

```
    float eventX = event.getX();
```

← 화면이 터치된 위치를 얻는다.

```
    float eventY = event.getY();
```

```
    switch (event.getAction()) {
```

```
        case MotionEvent.ACTION_DOWN:
```

```
            path.moveTo(eventX, eventY);
```

← 터치가 시작되면 경로에
시작점을 저장한다.

```
            return true;
```

```
        case MotionEvent.ACTION_MOVE:
```

```
            path.lineTo(eventX, eventY);
```

← 화면에서 손가락이 움직이면
경로에 직선을 저장한다.

```
            break;
```

```
        case MotionEvent.ACTION_UP:
```

```
            break;
```

```
        default:
```

```
            return false;
```

```
    }
```

```
    invalidate();
```

```
    return true;
```

```
}
```

```
}
```



멀티 터치 이벤트

- 멀티 터치란 여러 개의 손가락을 이용하여 화면을 터치하는 것으로 이미지를 확대/축소할 때 많이 사용된다.
- 여러 개의 포인터(손가락 또는 스타일러스)가 화면을 동시에 터치하면 다음과 같은 터치 이벤트들이 발생된다.





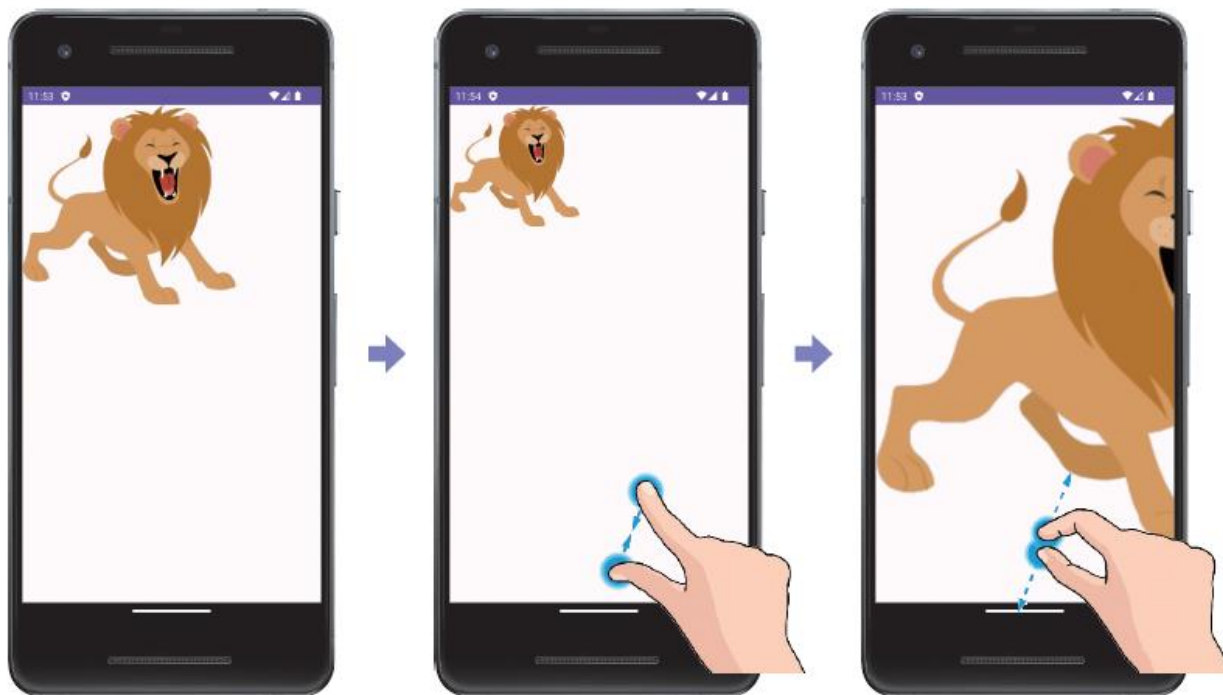
멀티 터치 이벤트

- **ACTION_DOWN**: 화면을 터치하는 첫 번째 포인터에 대하여 발생한다. 제스처 인식이 시작된다. 첫 번째 터치는 항상 **MotionEvent**에서 인덱스 0번에 저장된다.
- **ACTION_POINTER_DOWN**: 첫 번째 포인터 이외의 포인터에 대하여 발생된다. 포인터 데이터는 **getActionIndex()**가 반환하는 인덱스에 저장된다.
- **ACTION_MOVE**: 화면을 누르면서 이동할 때 발생한다.
- **ACTION_POINTER_UP**: 마지막 포인터가 아닌 다른 포인터가 화면에서 없어지면 발생된다.
- **ACTION_UP**: 화면을 떠나는 마지막 포인터에 대하여 발생된다.



예제: 핀치 줌

- 멀티 터치 of 가장 대표적인 예제는 이미지를 두 개의 손가락으로 확대하거나 축소하는 예제일 것이다. 이것을 핀치 줌(pinch zoom)이라고 한다.





예제: 핀치 줌

MainActivity.java

```
package kr.co.company.imagescale;

// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(new MyImageView(this));
    }
}
```



예제: 핀치 줌

MyImageView.java

```
package kr.co.company.imagescale;
```

```
// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.
```

```
public class MyImageView extends View {
```

```
    private Drawable image;
```

```
    private ScaleGestureDetector gestureDetector;
```

```
    private float scale = 1.0f;
```

```
    public MyImageView(Context context) {
```

```
        super(context);
```

```
        image = context.getResources().getDrawable(R.drawable.lion);
```

```
        setFocusable(true);
```

```
        image.setBounds(0, 0, image.getIntrinsicWidth(), image.getIntrinsicHeight());
```

```
        gestureDetector = new ScaleGestureDetector(context, new ScaleListener());
```

```
    }
```

제스처 인식기 객체를 생성한다.



예제: 핀치 zoom

```
@Override
protected void onDraw(Canvas canvas) {
    super.onDraw(canvas);
    canvas.save();
    canvas.scale(scale, scale);
    image.draw(canvas);
    canvas.restore();
}

@Override
public boolean onTouchEvent(MotionEvent event) {
    gestureDetector.onTouchEvent(event);
    invalidate();
    return true;
}
```

캔버스의 현재 상태를 저장한다.

캔버스에 신축 연산을 적용한다.
좀 더 자세한 내용은 다음 장을 참조한다.

캔버스의 상태를 복원한다.

제스처 인식기로 터치 이벤트를 보내서 처리한다.



예제: 핀치 줌

```
private class ScaleListener extends  
    ScaleGestureDetector.SimpleOnScaleGestureListener {
```

```
    @Override
```

```
    public boolean onScale(ScaleGestureDetector detector) {  
        scale *= detector.getScaleFactor();  
  
        if (scale < 0.1f)  
            scale = 0.1f;  
        if (scale > 10.0f)  
            scale = 10.0f;  
  
        invalidate();  
        return true;  
    }
```

```
}
```

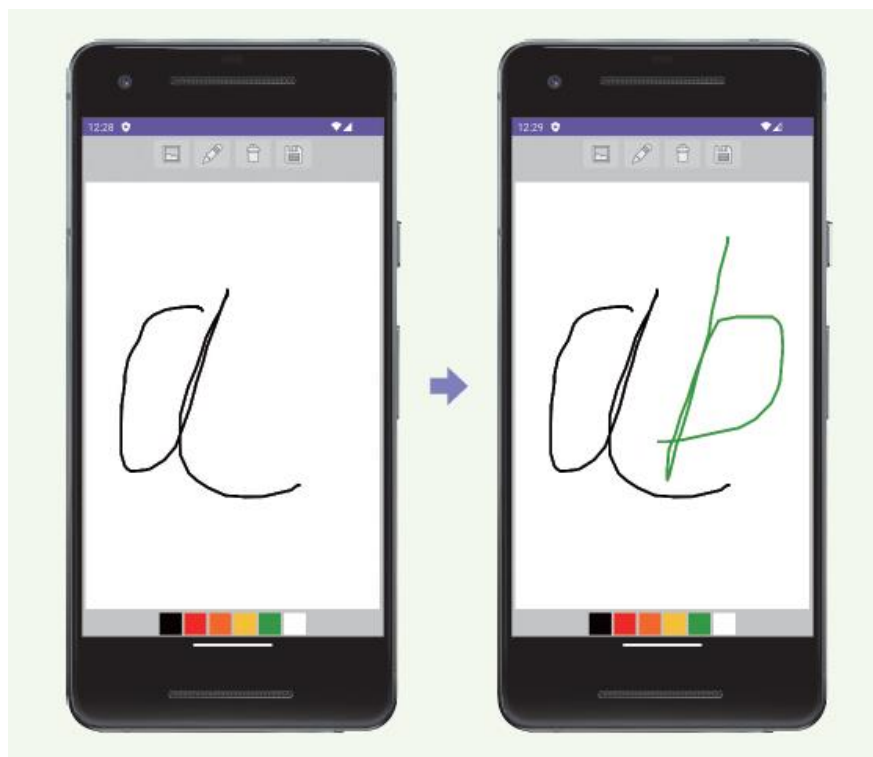
```
}
```

신축 연산이 감지되었으면
호출된다.



Lab: 그림판 예제 작성

- 우리는 예제에서 터치 이벤트를 처리하여 간단한 그림판 프로그램을 만들어 본 적이 있다. 이것을 확장하여 좀 더 정교한 그림판 애플리케이션을 작성해보자.





Lab: 그림판 예제 작성

SingleTouchView.java

```
package kr.co.company.mydraw;

// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.

public class SingleTouchView extends View {
    private Paint paint = new Paint();
    private Path path = new Path();
    private int paintColor = 0xFF000000;
    private Paint canvasPaint;
    private Canvas drawCanvas;
    private Bitmap canvasBitmap;

    public SingleTouchView(Context context, AttributeSet attrs) {
        super(context, attrs);
        paint.setAntiAlias(true);
        paint.setStrokeWidth(10f);
        paint.setColor(paintColor);
    }
}
```



Lab: 그림판 예제 작성

```
    paint.setStyle(Paint.Style.STROKE);  
    paint.setStrokeJoin(Paint.Join.ROUND);  
}
```

@Override

```
protected void onSizeChanged(int w, int h, int oldw, int oldh) {  
    super.onSizeChanged(w, h, oldw, oldh);  
    canvasBitmap = Bitmap.createBitmap(w, h, Bitmap.Config.ARGB_8888);  
    drawCanvas = new Canvas(canvasBitmap);  
    canvasPaint = new Paint(Paint.DITHER_FLAG);  
}
```

비트맵을 생성하고, 이것을
캔버스와 연결한다. 캔버스에
무언가 그려지면 비트맵에도
그려진다.

@Override

```
protected void onDraw(Canvas canvas) {  
    canvas.drawBitmap(canvasBitmap, 0, 0, canvasPaint);  
    canvas.drawPath(path, paint);  
}
```

비트맵을 그린 후에 현재의 경로를 그린다.



Lab:

@Override

public boolean onTouchEvent(MotionEvent event) {

float touchX = event.getX();

float touchY = event.getY();

switch (event.getAction()) {

case MotionEvent.ACTION_DOWN:

path.moveTo(touchX, touchY);

break;

case MotionEvent.ACTION_MOVE:

path.lineTo(touchX, touchY);

break;

case MotionEvent.ACTION_UP:

drawCanvas.drawPath(path, paint);

path.reset();

break;

default:

return false;

}

invalidate();

return true;

}

public void setColor(String newColor) {

invalidate();

paintColor = Color.parseColor(newColor);

paint.setColor(paintColor);

}

}

터치이벤트가 발생하면 현재의 경로에
직선으로 추가한다.

손가락을 화면에서 떼는 순간, 현재까지의 경로를
캔버스에 그리고, 경로는 리셋한다.



Lab: 그림판 예제 작성

activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:background="#FFCCCC"
    tools:context=".MainActivity" >
```

```
    <LinearLayout        android:orientation="horizontal" >
        <ImageButton      android:src="@android:drawable/ic_menu_gallery" />
        <ImageButton      android:src="@android:drawable/ic_menu_edit" />
        <ImageButton      android:src="@android:drawable/ic_menu_delete" />
        <ImageButton      android:src="@android:drawable/ic_menu_save" />
    </LinearLayout>
```

```
    <kr.co.company.singletouch.SingleTouchView
        android:id="@+id/drawing"
        android:layout_width="fill_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:background="#FFFFFF" />
```



Lab: 그림판 예제 작성

```
<LinearLayout android:orientation="vertical" >
  <LinearLayout android:orientation="horizontal" >
    <ImageButton
      android:background="#FF000000"
      android:onClick="clicked"
      android:tag="#FF000000" />
    <ImageButton
      android:background="#FFFF0000"
      android:onClick="clicked"
      android:tag="#FFFF0000" />
    <ImageButton
      android:background="#FFFF6600"
      android:onClick="clicked"
      android:tag="#FFFF6600" />
    <ImageButton
      android:background="#FFFFCC00"
      android:onClick="clicked"
      android:tag="#FFFFCC00" />
    <ImageButton
      android:background="#FF009900"
      android:onClick="clicked"
      android:tag="#FF009900" />
    <ImageButton
      android:background="#FFFFFFF" />
  </LinearLayout>
  <Canvas />
</LinearLayout>
```





Lab:

MainActivity.java

```
package kr.co.company.mydraw;

// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.

public class MainActivity extends AppCompatActivity {
    private SingleTouchView drawView;
    private ImageButton currPaint;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        drawView = (SingleTouchView) findViewById(R.id.drawing);
        LinearLayout paintLayout = (LinearLayout) findViewById(R.id.paint_colors);
        currPaint = (ImageButton) paintLayout.getChildAt(0);
    }

    public void clicked(View view) {
        if (view != currPaint) {
            String color = view.getTag().toString();
            drawView.setColor(color);
            currPaint = (ImageButton) view;
        }
    }
}
```



이미지 표시하는 방법

- ① 레이아웃 파일에 이미지를 표시하기
- ② 캔버스에서 이미지 그려주기



배경도 하나의 큰 이미지
이다.

게임에서는 거의 모든 것이
이미지로 처리된다.



이미지를 불러오는 방법

방법	설명
리소스로 표시하는 방법	먼저 이미지 파일을 프로젝트의 리소스로 추가하여야 한다. 예를 들어서 dog.jpg 파일을 프로젝트의 res/drawable 폴더에 추가한다. 이미지를 Ctrl+C로 복사하고 /res/drawable 폴더 위에서 Ctrl+V하면 된다. 이미지가 리소스로 추가되면 소스 코드나 XML에서 식별자를 통하여 참조될 수 있다. 안드로이드에서는 확장자를 제외한 파일의 이름을 식별자로 사용한다. 예를 들어서 dog.jpg 파일은 R.drawable.dog으로 참조된다.
이미지 파일을 직접 읽는 방법	이미지 파일 경로를 이용하여 비트맵으로 읽어오는 방법이다. <pre>Bitmap bitmap = BitmapFactory.decodeFile(filePath.getAbsolutePath()); cnavas.drawBitmap(bitmap, x, y, null);</pre>



이미지를 직접 그려주는 방법

- 소스 코드에서 이미지를 불러올 때는 **BitmapFactory** 클래스의 **decode Resource()**를 사용하는 것이 가장 간편하다.

비트맵 읽기

`BitmapFactory.decodeResource()`



비트맵 그리기

`drawBitmap(Bitmap bitmap, ...)`



이미지를 직접 그려주는 방법

```
Bitmap b = BitmapFactory.decodeResource(getResource(), R.drawable.android);
```

이미지 데이터를 포함하는 애플리케이션 리소스 객체

이미지의 리소스 식별자

```
void drawBitmap(Bitmap bitmap, float left, float top, Paint paint);
```

화면에 그릴 비트맵

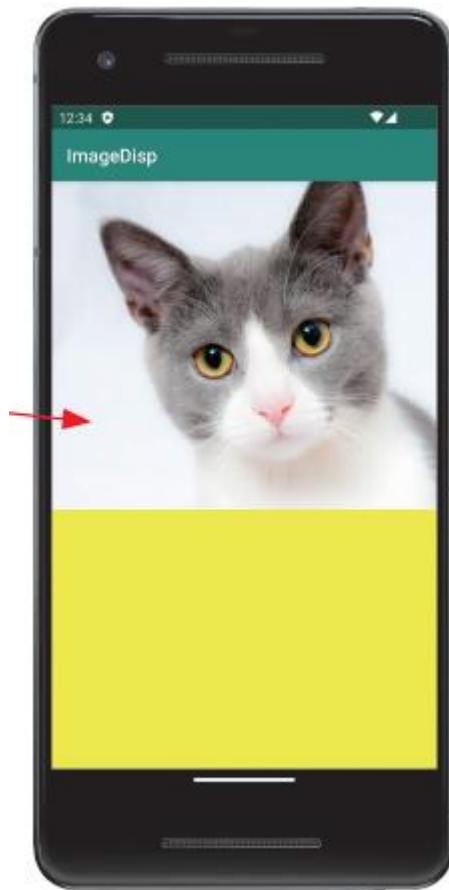
이미지가 그려지는 위치

변환 행렬이 들어 있는 Paint 객체



예제: 화면에 이미지 그리기

- 이미지 파일을 화면에 표시하는 간단한 예제를 작성하여 보자.





예제:

MainActivity.java

```
package kr.co.company.imagedisp;  
// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.
```

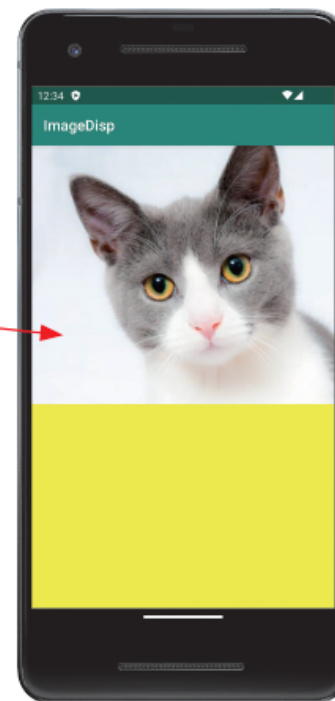
```
class MyView extends View {  
    public MyView(Context context) {  
        super(context);  
        setBackgroundColor(Color.YELLOW);  
    }
```

```
    @Override  
    protected void onDraw(Canvas canvas) {  
        Paint paint = new Paint();  
        Bitmap b = BitmapFactory.decodeResource(getResources(),  
            R.drawable.cat);  
        canvas.drawBitmap(b, 0, 0, null);  
    }  
}
```

이미지를 읽어서 비트맵으로 만든다.

이미지를 캔버스에 그린다.

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        MyView w = new MyView(this);  
        setContentView(w);  
    }  
}
```



비트맵의 확대 및 축소(크기만 변경)

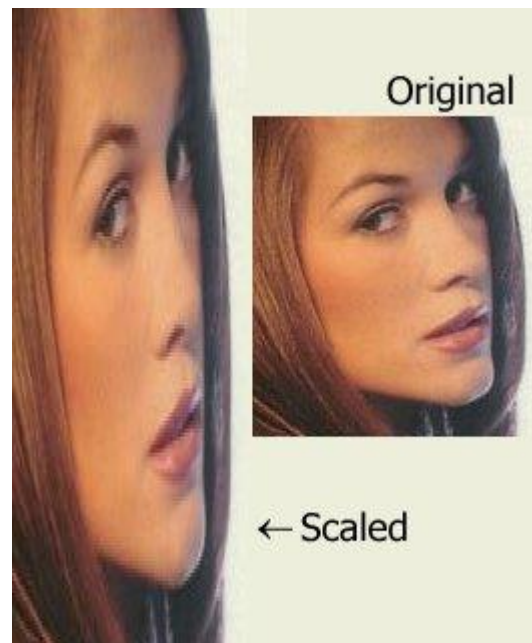
...

```
Bitmap b = BitmapFactory.decodeResource(getResource(), R.drawable.cat);
```

```
Bitmap sb= Bitmap.createScaledBitmap(b, 60, 80, false);
```

...

60×80 크기의 새로운
비트맵을 생성한다.





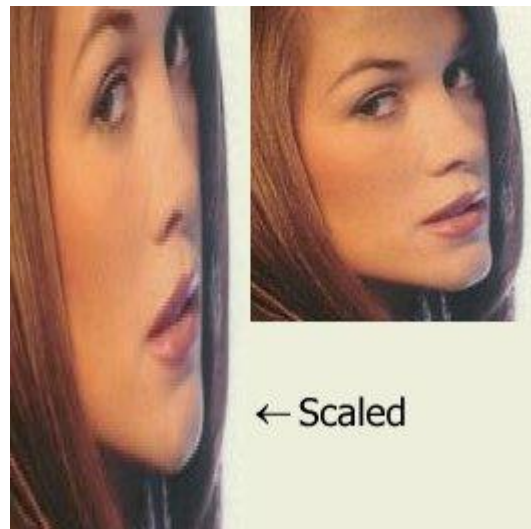
비트맵의 확대 및 축소(변환 행렬 이용)

```
...  
Matrix m = new Matrix();  
m.preScale(1, -1);  
Bitmap b = BitmapFactory.decodeResource(getResources(), R.drawable.cat);  
Bitmap mb=Bitmap.createBitmap(b, 0, 0, b.getWidth(), b.getHeight(), m, false);  
...
```

변환 행렬을 생성한다.

x값에 1을 곱하고 y값에 -1을 곱하는 변환 행렬을 만든다.

변환 행렬 m이 적용된 새로운 비트맵을 생성한다.





예제: 상하 반전 이미지

- house.png 파일을 읽어서 상하 반전시킨 이미지를 생성해서 화면에 출력한다. 이어서 원 파일을 200×200 으로 확대한 비트맵을 화면 위치 (100, 100)에 출력한다.





예제: 상하 반전 이미지

MainActivity.java

...

```
protected void onDraw(Canvas canvas) {
```

```
    Paint paint = new Paint();
```

```
    Matrix m= new Matrix();
```

```
    m.preScale(1, -1);
```

← 상하반전 변환 행렬

```
    Bitmap b = BitmapFactory.decodeResource(getResources(), R.drawable.house);
```

```
    Bitmap mb = Bitmap.createBitmap(b, 0, 0, b.getWidth(), b.getHeight(), m, false);
```

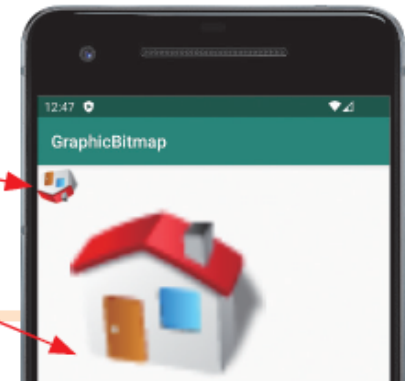
```
    Bitmap sb = Bitmap.createScaledBitmap(b, 600, 600, false);
```

```
    canvas.drawBitmap(mb, 0, 0, null);
```

```
    canvas.drawBitmap(sb, 100, 100, null);
```

```
}
```

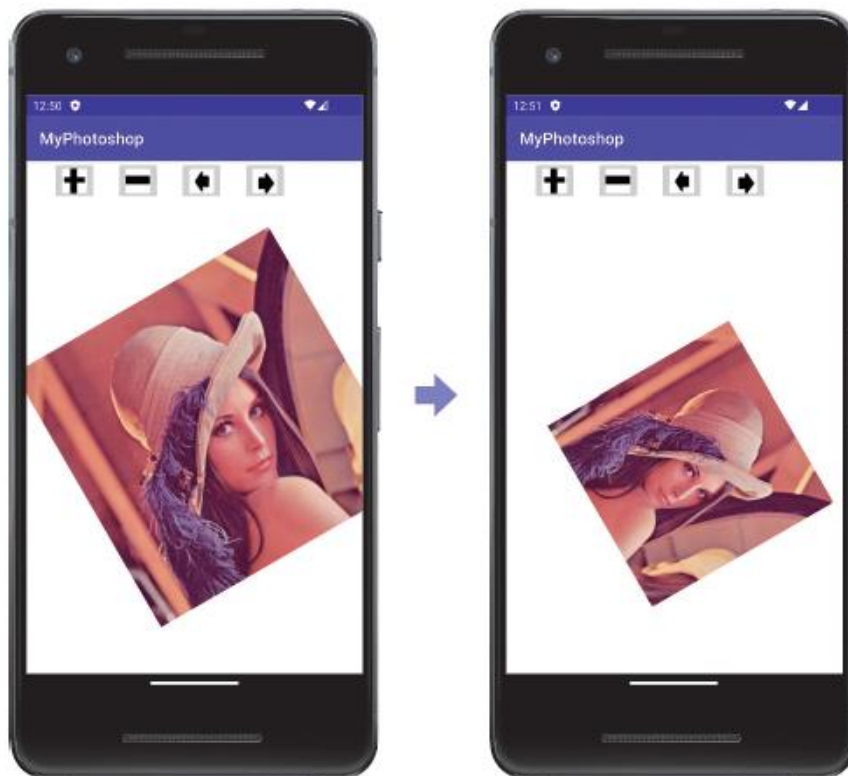
...





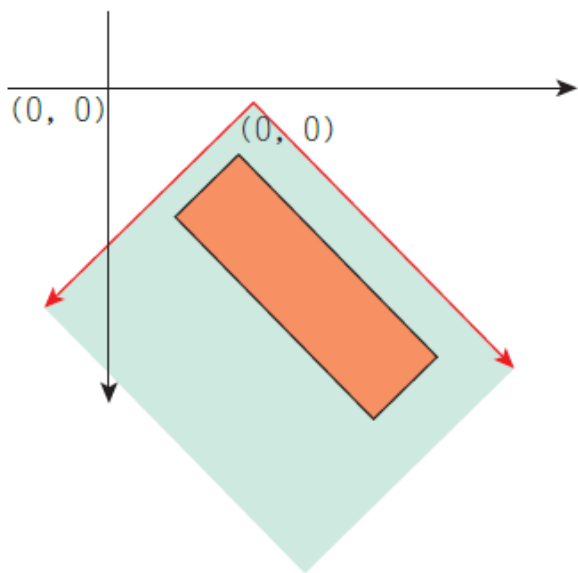
예제: 미니 포토샵 만들기

- 안드로이드의 영상 처리 기능을 이용하여서 포토샵과 유사한 프로그램을 작성해보자.



도형을 회전하지 않고 캔버스를 회전

```
canvas.scale(scaleX, scaleY, centerX, centerY); // 캔버스 축척을 바꾼다.  
canvas.rotate(rotateAngle, centerX, centerY); // 캔버스를 회전시킨다.
```





예제: 미니 포토샵 만들기

activity_main.xml

```
<androidx.constraintlayout.widget.ConstraintLayout>
```

```
<ImageButton
```

```
    android:id="@+id/imageButton"  
    android:onClick="expand"  
    app:srcCompat="@drawable/plus" />
```

```
<ImageButton
```

```
    android:id="@+id/imageButton2"  
    android:onClick="shrink"  
    app:srcCompat="@drawable/minus" />
```

```
<ImageButton
```

```
    android:id="@+id/imageButton3"  
    android:onClick="rotateLeft"  
    app:srcCompat="@drawable/left" />
```

```
<ImageButton
```

```
    android:id="@+id/imageButton4"  
    android:onClick="rotateRight"  
    app:srcCompat="@drawable/right" />
```





예제: 미니 포토샵 만들기

MainActivity.java

```
package kr.co.company.myphotoshop;
```

```
public class MainActivity extends AppCompatActivity {  
    float scaleX = 1.0f, scaleY = 1.0f, rotateAngle;  
    MyView displayView;
```

```
    public class MyView extends View {  
        public MyView(MainActivity context) {  
            super(context);  
        }
```

```
        @Override
```

```
        protected void onDraw(Canvas canvas) {  
            super.onDraw(canvas);  
            Paint paint = new Paint();
```

```
            int centerX = getWidth() / 2;
```

```
            int centerY = getHeight() / 2;
```

```
            canvas.scale(scaleX, scaleY, centerX, centerY);
```

```
            canvas.rotate(rotateAngle, centerX, centerY);
```

현재 설정된 축척과 회전각도로
비트맵을 화면에 그린다.



예제: 미니 포토샵 만들기

```
        Bitmap picture = BitmapFactory.decodeResource(getResources(),
            R.drawable.lena);
        canvas.drawBitmap(picture, 0, 0, paint);
    }
}
```

@Override

```
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    LinearLayout layout = (LinearLayout) findViewById(R.id.imageDisplay);
    displayView = new MyView(this);
    layout.addView(displayView);
}
```

레이아웃에서 선형 레이아웃을
찾아서, 거기에 커스텀 뷰를 추
가한다.

```
public void expand(View view) {
    scaleX += 0.3;
}
```



예제: 미니 포토샵 만들기

```
        scaleY += 0.3;  
        displayView.invalidate();  
    }
```

```
public void shrink(View view) {
```

← 그림을 축소해서 그린다.

```
    scaleX -= 0.3;  
    scaleY -= 0.3;  
    displayView.invalidate();  
}
```

```
public void rotateLeft(View view) {
```

← 그림을 왼쪽으로 회전시켜 그린다.

```
    rotateAngle -= 30;  
    displayView.invalidate();  
}
```

```
public void rotateRight(View view) {  
    rotateAngle += 30;  
    displayView.invalidate();  
}
```

```
}
```



드로워블 애니메이션

- 영화 필름처럼 여러 개의 이미지가 순서대로 재생되어서 생성되는 전통적인 애니메이션





드로워블 애니메이션

- 애니메이션을 구성하는 프레임들을 나열하는 XML 파일을 생성



전체
구조

```
<?xml version="1.0" encoding="utf-8"?>
<animation-list xmlns:android="http://schemas.android.com/apk/res/android"
    android:oneshot="false" >

    <item
        android:drawable="@drawable/rocket1"
        android:duration="100"/>

    <item
        android:drawable="@drawable/rocket2"
        android:duration="100"/>

    <item
        android:drawable="@drawable/rocket3"
        android:duration="100"/>

</animation-list>
```

← 이미지 #1

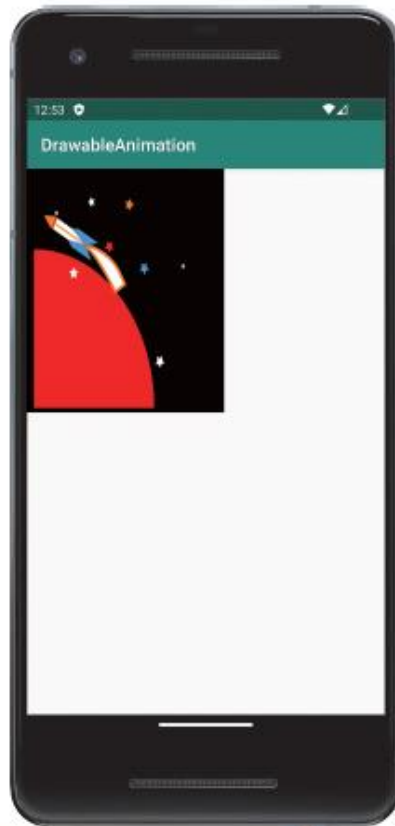
← 이미지 #2

← 이미지 #3



예제: 애니메이션 만들어보기

- 이 애니메이션은 3개의 프레임으로 이루어져 있다. `android:oneshot` 속성을 `false`로 설정하였기 때문에 무한 반복될 것이다.





예제: 애니메이션 만들어보기

MainActivity.java

```
package kr.co.company.drawableanimation;  
// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.
```

```
public class MainActivity extends AppCompatActivity {  
    AnimationDrawable rocketAnimation;
```

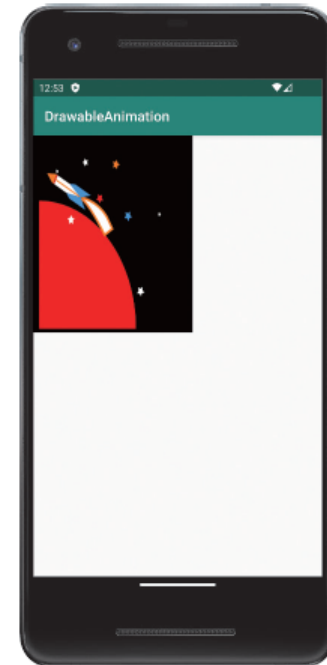
```
    public void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        ImageView rocketImage = (ImageView) findViewById(R.id.rocket_image);  
        rocketImage.setBackgroundResource(R.drawable.rocket);  
        rocketAnimation = (AnimationDrawable) rocketImage.getBackground();  
    }
```

애니메이션 리소스를 이미지
뷰의 배경으로 설정한다.

```
    public boolean onTouchEvent(MotionEvent event) {  
        if (event.getAction() == MotionEvent.ACTION_DOWN) {  
            rocketAnimation.start();  
            return true;  
        }  
        return super.onTouchEvent(event);  
    }  
}
```

애니메이션 객체를 얻는다.

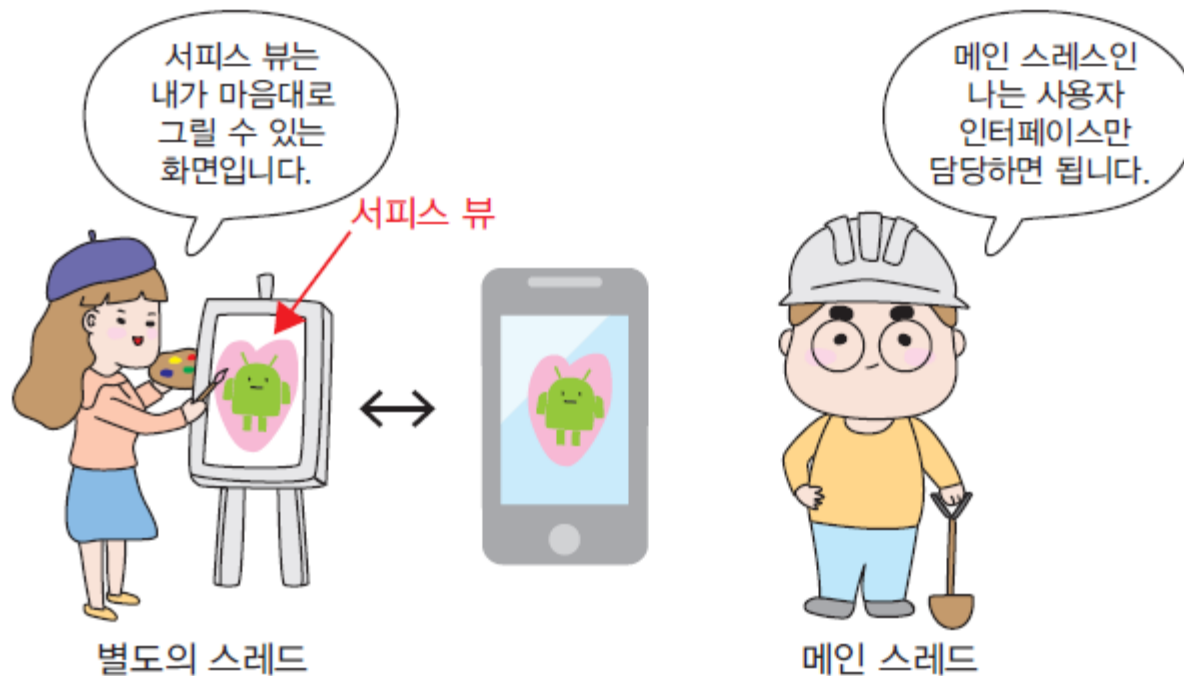
화면을 터치하면 애니메이션을 시작한다.





서피스 뷰

- 서피스뷰는 사용자 인터페이스와는 별도로 애플리케이션에게 그림을 그릴 수 있는 화면을 제공





서피스 뷰의 구조



전체
구조

```
class MyView extends SurfaceView implements SurfaceHolder.Callback {
```

```
    public void surfaceCreated(SurfaceHolder holder) {  
        // 서피스가 준비되었으므로 스레드를 시작한다.  
        ...  
    }
```

← 서피스 뷰의 화면이 생성되면 호출된다. 이 메소드가 호출된 후부터 화면에 그림을 그릴 수 있다. 통상적으로 이 메소드 안에서 그림을 그리는 스레드를 생성한다.

```
    public void surfaceDestroyed(SurfaceHolder holder) {  
        // 서피스가 소멸되었으므로 스레드를 종료한다.  
        ...  
    }
```

← 서피스 뷰의 화면이 파괴되기 직전에 호출된다. 이 메소드가 호출된 후에는 더 이상 화면에 그림을 그릴 수 없다. 통상적으로 이 메소드에서 그림을 그리는 스레드를 종료한다.

```
    public void surfaceChanged(SurfaceHolder holder, int format, int width,  
        int height) {  
        // 서피스가 변경  
        ...  
    }
```

← 화면의 구조적인 변경 (포맷이나 크기)이 있는 경우에 호출된다.

```
}
```



스레드를 정의한다.



전체
구조

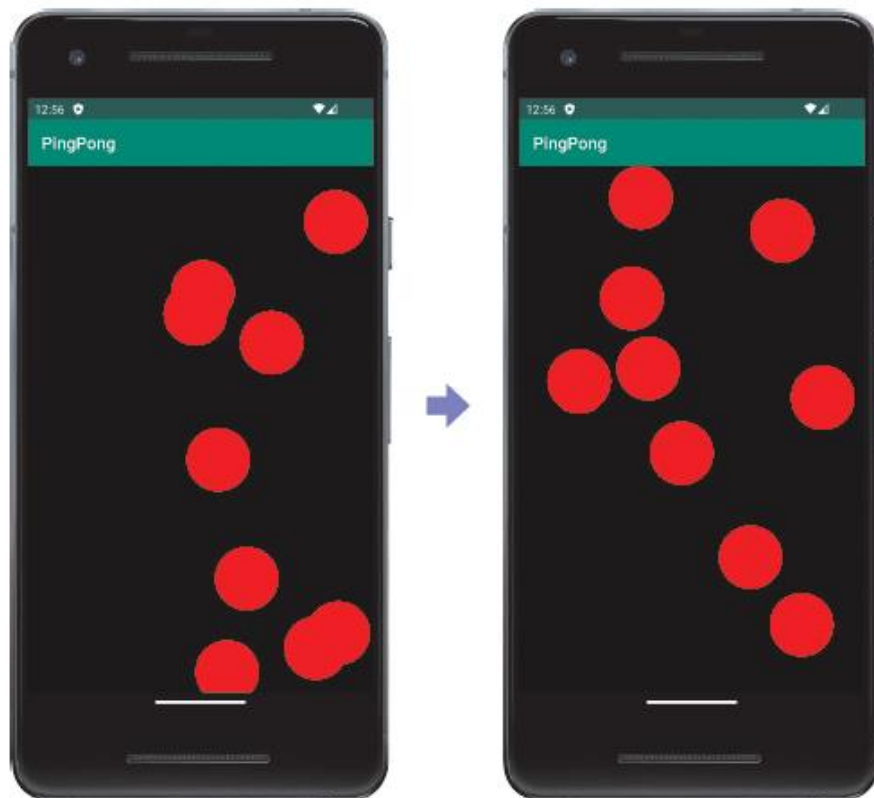
```
class MyThread extends Thread {  
    SurfaceHolder holder;  
    ...  
    public void run()  
    {  
        canvas = holder.lockCanvas();  
        // 캔버스에 그림을 그린다.  
        ...  
        ...  
        holder.unlockCanvasAndPost(canvas);  
    }  
}
```

이 메소드가 호출되면 서피스 뷰 안의 픽셀들을 변경할 수 있다. 반환되는 캔버스에 그림을 그리면 서피스 뷰 안의 비트맵에 그려진다.

이 메소드가 호출되면 서피스 뷰 안의 픽셀의 변경이 종료되고 서피스 뷰의 현재 픽셀값이 실제 장치의 화면에 보이게 된다. 그러나 서피스 뷰의 내용은 보존되지 않는다. 즉 다시 lockCanvas()가 호출되었을 때 서피스 뷰의 내용이 그대로 보존된다는 보장은 없다.

예제: 랜덤하게 움직이는 공 그리기

- 랜덤하게 움직이는 공을 그리는 애플리케이션을 작성하여 보자.



예제: 랜덤하게 움직이는 공 그리기

MainActivity.java

```
package kr.co.company.pingpong;

// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.

public class MainActivity extends AppCompatActivity {
    MySurfaceView view;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        view = new MySurfaceView(this); <-----서피스 뷰를 액티비티의 화면으로 설정한다.
        setContentView(view);
    }

    @Override
    protected void onPause() {
        super.onPause();
    }

    protected void onSaveInstanceState(Bundle outState) {
        super.onSaveInstanceState(outState);
    }
}
```

예제: 랜덤하게 움직이는 공 그리기

MySurfaceView.java

```
package kr.co.company.pingpong;

// 소스만 입력하고 Alt+Enter를 눌러서 import 문장을 자동으로 생성한다.

// 움직이는 공을 나타내는 클래스
class Ball {
    int x, y, xInc = 1, yInc = 1;    // xInc와 yInc는 한 번에 움직이는 거리이다.
    int diameter;    // 공의 반지름
    static int WIDTH = 1080, HEIGHT = 1920;    // 움직이는 공간의 크기

    public Ball(int d) {    // 생성자
        this.diameter = d;

        // 볼의 위치를 랜덤하게 설정
        x = (int) (Math.random() * (WIDTH - d) + 3);
        y = (int) (Math.random() * (HEIGHT - d) + 3);

        // 한번에 움직이는 거리도 랜덤하게 설정
        xInc = (int) (Math.random() * 30 + 1);
        yInc = (int) (Math.random() * 30 + 1);
    }
}
```

예제: 랜덤하게 움직이는 공 그리기

```
// 여기서 공을 그린다.  
public void paint(Canvas g) {  
    Paint paint = new Paint();  
  
    // 벽에 부딪히면 반사하게 한다.  
    if (x < 0 || x > (WIDTH - diameter))  
        xInc = -xInc;  
    if (y < 0 || y > (HEIGHT - diameter))  
        yInc = -yInc;  
  
    // 볼의 좌표를 갱신한다.  
    x += xInc;  
    y += yInc;  
  
    // 볼을 화면에 그린다.  
    paint.setColor(Color.RED);  
    g.drawCircle(x, y, diameter, paint);  
}  
}
```

예제: 랜덤하게 움직이는 공 그리기

```
// 서피스 뷰 정의
```

```
public class MySurfaceView extends SurfaceView implements
    SurfaceHolder.Callback {
    public Ball basket[] = new Ball[10]; // Ball의 배열 선언
    private MyThread thread; // 스레드 참조 변수
```

```
    public MySurfaceView(Context context) { // 생성자
        super(context);
```

```
        SurfaceHolder holder = getHolder(); // 서피스 뷰의 홀더를 얻는다.
        holder.addCallback(this); // 콜백 메소드를 처리한다.
```

```
        thread = new MyThread(holder); // 스레드를 생성한다.
```

```
        // Ball 객체를 생성하여서 배열에 넣는다.
```

```
        for (int i = 0; i < 10; i++)
            basket[i] = new Ball(20);
```

```
    }
```

```
    public MyThread getThread() {
        return thread;
    }
```

예제: 랜덤하게 움직이는 공 그리기

```
public void surfaceCreated(SurfaceHolder holder) {
    // 스레드를 시작한다.
    thread.setRunning(true);
    thread.start();
}

public void surfaceDestroyed(SurfaceHolder holder) {
    boolean retry = true;

    // 스레드를 중지시킨다.
    thread.setRunning(false);
    while (retry) {
        try {
            thread.join(); // 메인 스레드와 합친다.
            retry = false;
        } catch (InterruptedException e) {
        }
    }
}
```

예제: 랜덤하게 움직이는 공 그리기

```
public void surfaceChanged(SurfaceHolder holder, int format, int width,
    int height) {
    // Ball.WIDTH = width;
    // Ball.HEIGHT = height;

}
```

// 스레드를 내부 클래스로 정의한다.

```
public class MyThread extends Thread {

    private boolean mRun = false;
    private SurfaceHolder mSurfaceHolder;

    public MyThread(SurfaceHolder surfaceHolder) {
        mSurfaceHolder = surfaceHolder;
    }
}
```



예제:

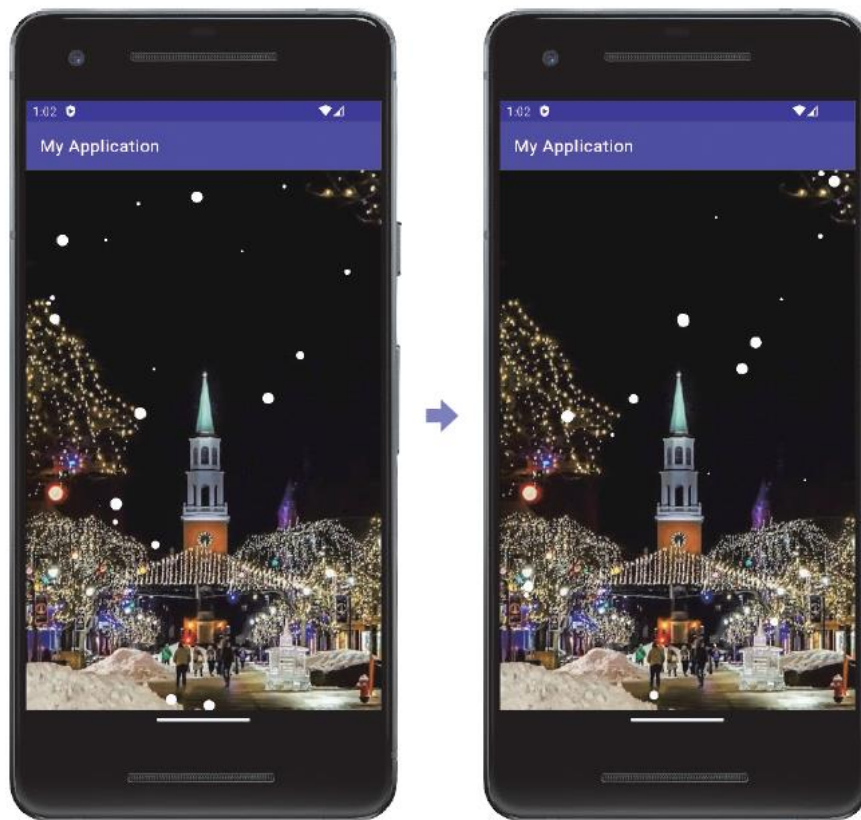
```
@Override
public void run() {
    while (mRun) {
        Canvas c = null;
        try {
            c = mSurfaceHolder.lockCanvas(null); // 캔버스를 얻는다.
            c.drawColor(Color.BLACK); // 캔버스의 배경을 지운다.
            synchronized (mSurfaceHolder) {
                for (Ball b : basket) { // basket의 모든 원소를 그린다.
                    b.paint(c);
                }
            }
        } finally {
            if (c != null) {
                // 캔버스의 로킹을 푼다.
                mSurfaceHolder.unlockCanvasAndPost(c);
            }
        }
        // try { Thread.sleep(100); } catch (InterruptedException e) { }
    }
}

public void setRunning(boolean b) {
    mRun = b;
}
}
```



Coding Challenge: 눈 오는 풍경 만들기

- 본문의 PingPong 프로그램을 참조하여 캔버스 위에 어떤 그림을 표시한 후에 그래픽으로 눈을 내리게 하는 앱을 작성해보자. 눈의 크기나 개수를 조절 가능하도록 한다.





Q & A

